

A&I

android and ios develop.

학습 정리

2026 A&I 4th

구승민

목차

1. 학습 흐름 정리

- 2~6주차 학습 내용 복습

2. 주차별 정리

- 각 주차의 주요 개념 환기

3. 관통 개념 정리

- 주차를 가로지르는 핵심 개념 모으기

세션 안내

- 2~6주차에서 배운 것을 개념 수준으로 한 번에 정리
- 중간에 빠진 부분이 있어도 전체 그림을 따라올 수 있도록 하는 시간
- 각 주차가 어떻게 이어지는지만 확인

전체 흐름

- 2주차부터 6주차까지 학습한 내용을 다시 리뷰
- 학습하는 법 → 데이터 다루기 → 관계로 추천 → 순서로 추천 → 자동화·관측
- 모델 학습과 시스템을 구축하기 위한 기초 과정

주차 간 연결

- 임베딩(4주차)이 5주차 SASRec에서 그대로 다시 쓰임
- 내적으로 유사도를 재는 아이디어(4주차)가 5주차 예측 점수로 이어짐
- 5주차에서 만든 SASRec 학습이 6주차에서 자동 파이프라인으로 묶임

2. 주차별 정리

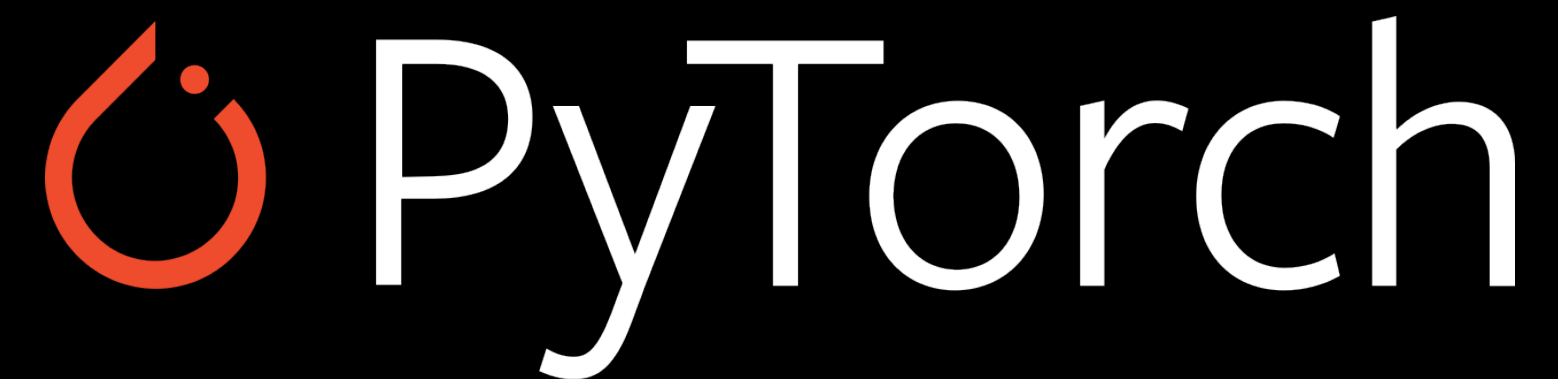
- 각 주차의 핵심 개념만 환기

2주차 PyTorch

- 기본적인 연산 방법 학습

데이터의 표현과 차원

- **Tensor** : 숫자를 담는 다차원 배열, 모델이 다루는 기본 데이터 단위
- **자동미분(Autograd)** : 손실이 줄어드는 방향을 컴퓨터가 자동 계산
- **반복 학습** : 예측 → 오차 확인 → 거꾸로 조정(역전파) → 반복
- **학습이란** 손실을 줄이도록 숫자(파라미터)를 조금씩 고치는 일

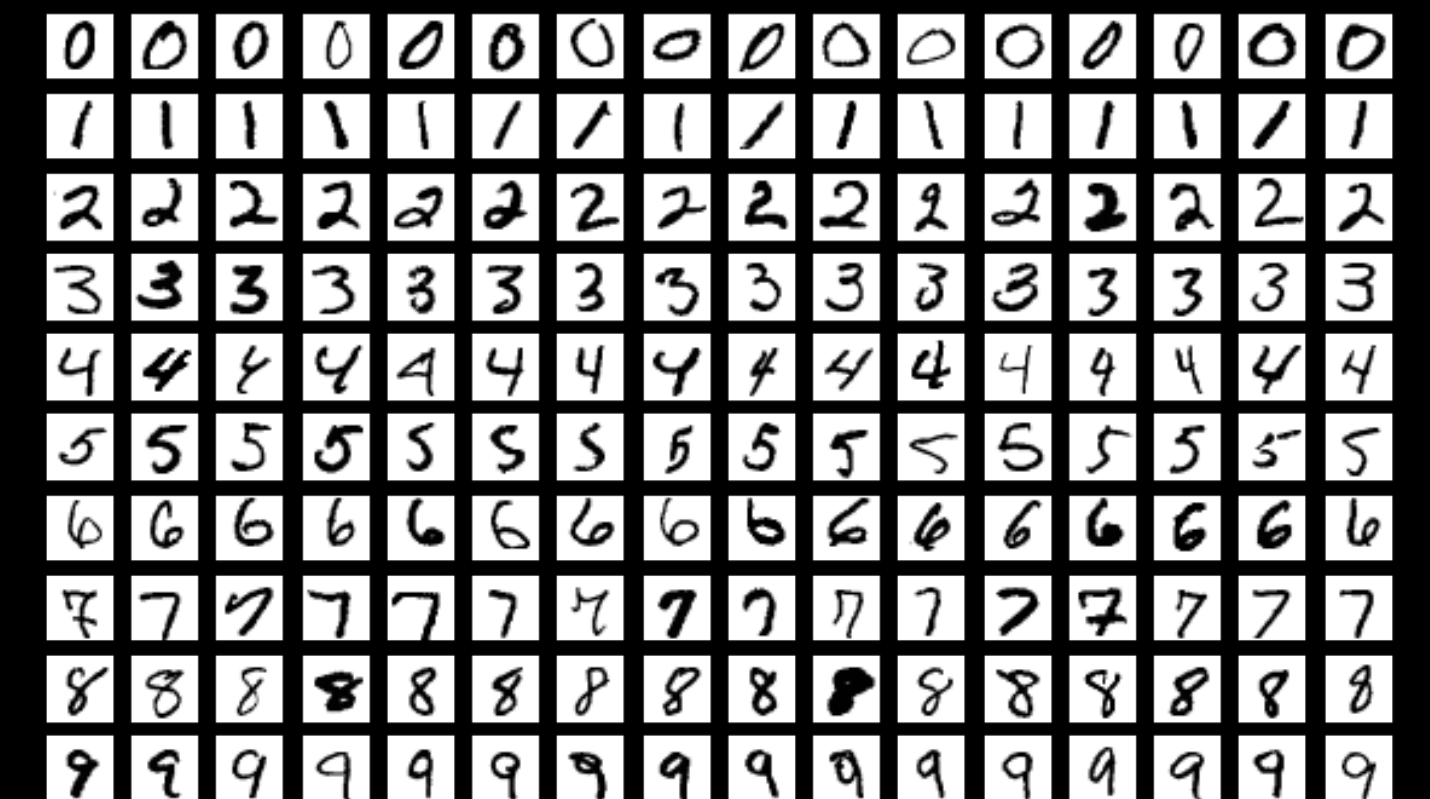


3주차 MNIST, EDA, 전처리

- 실제 데이터를 통한 연산 적용

데이터셋과 탐색 과정

- **MNIST** : 손글씨 숫자(0~9) 이미지, 입문용 표준 데이터셋
- **EDA** : 모델에 넣기 전 데이터가 어떻게 생겼는지 먼저 살펴보기
- 전처리·특징 공학 : 모델이 학습하기 좋은 형태로 데이터를 다듬기
- "좋은 데이터가 좋은 모델을 만든다" -> 데이터가 결국 모델의 성능



4주차 협업 필터링

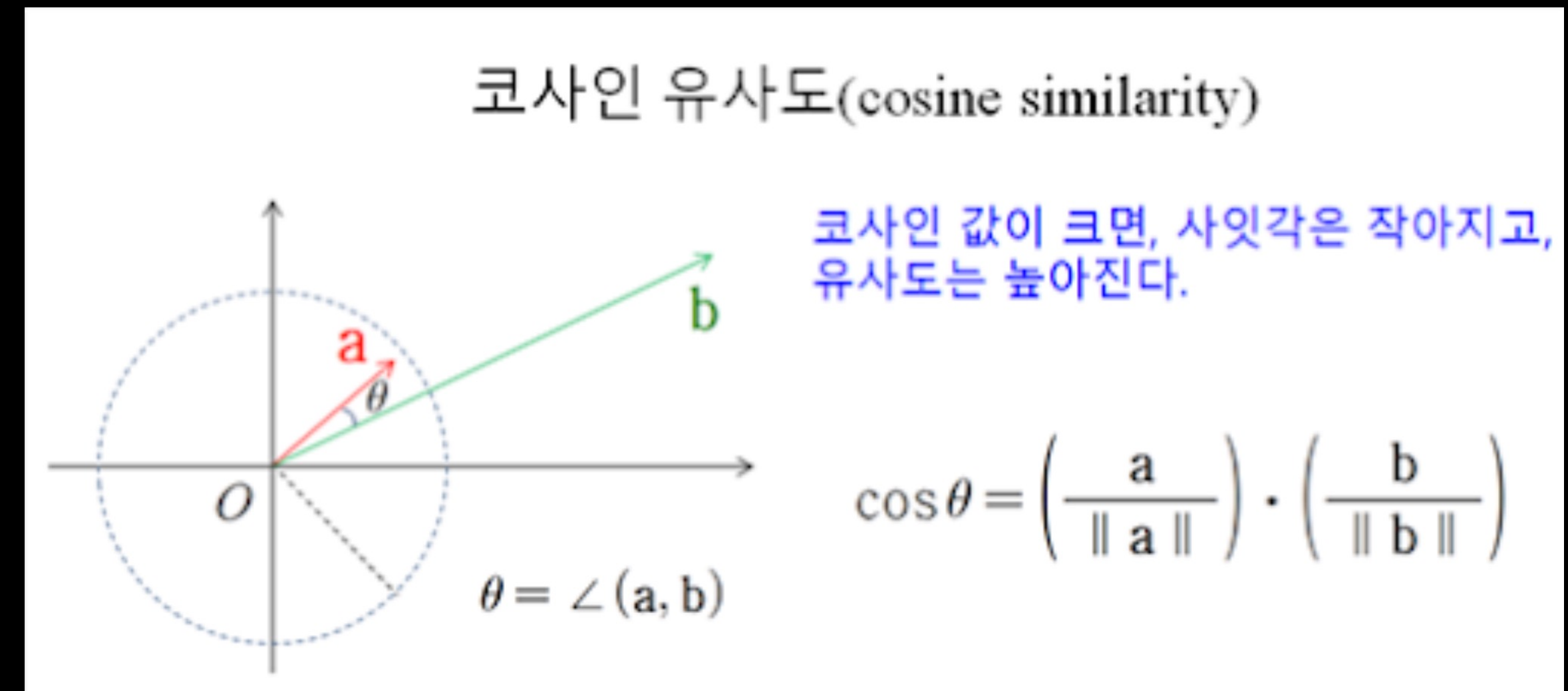
- 사용자와 아이템의 관계를 학습

평점 행렬과 데이터

- 추천의 출발점은 사용자 × 아이템 평점 행렬 (누가 무엇에 몇 점)
- 대부분 칸이 빈 희소 행렬 — 사람은 일부 영화만 보기 때문
- MovieLens — 943명이 1,682편에 매긴 평점으로 구성

임베딩과 코사인 유사도

- 임베딩: 사용자·아이템을 의미를 담은 숫자 벡터로 (비슷하면 가까이)
- 코사인 유사도: 두 벡터가 이루는 각도로 닮은 정도를 잴 (0도에 가까울수록 비슷)
- 방향이 비슷한 벡터 = 취향이 닮은 사용자/아이템



이웃 선정과 추천

- 이웃(kNN) — 나와 취향이 가장 닮은 상위 k명을 고름
- 그 이웃들이 좋아한 것을 모아 아직 안 본 아이템을 추천 (Top-K)
- User-based(비슷한 사람) vs Item-based(비슷한 아이템) 두 관점



한계와 행렬 분해

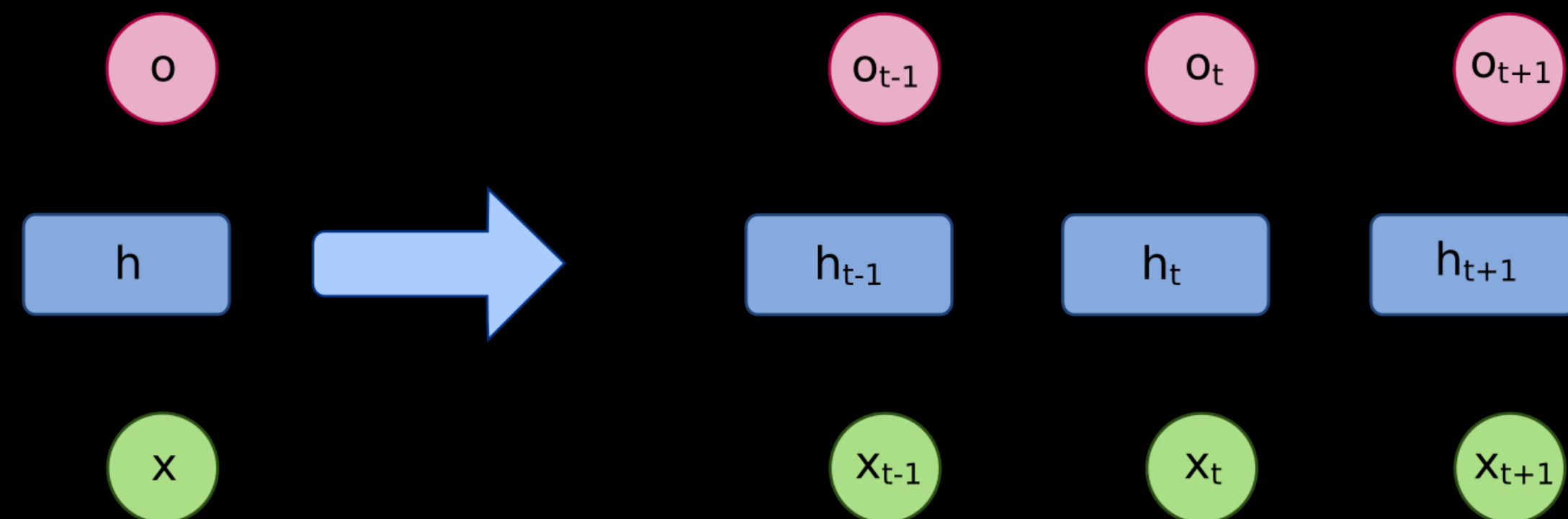
- 희소성 : 빈칸이 많아 님은 사용자를 찾기 어려움
- 확장성 : 커지면 계산량 폭증
- 행렬 분해(MF) : 큰 평점표를 두 작은 표의 곱으로 근사해 빈칸을 추정
- 가장 큰 한계
 - 평점을 순서 없는 집합으로만 인식

5주차 SASRec

- 연속적인 정보를 예측하는 모델

순차 추천의 필요성

- 1화를 본 사람에게 2화 -> **직전 행동**이 다음 행동의 강한 영향
- **Markov Chain** : 직전 한두 개만 봄 (단순하지만 **근시안적**)
- **RNN** : 앞에서부터 차례로 읽어 기억을 쌓음 (멀리 보지만 **느림**)
- **SASRec** : 둘의 장점(짧은 단서 + 장기 기억 + 빠른 병렬 처리)을 함께

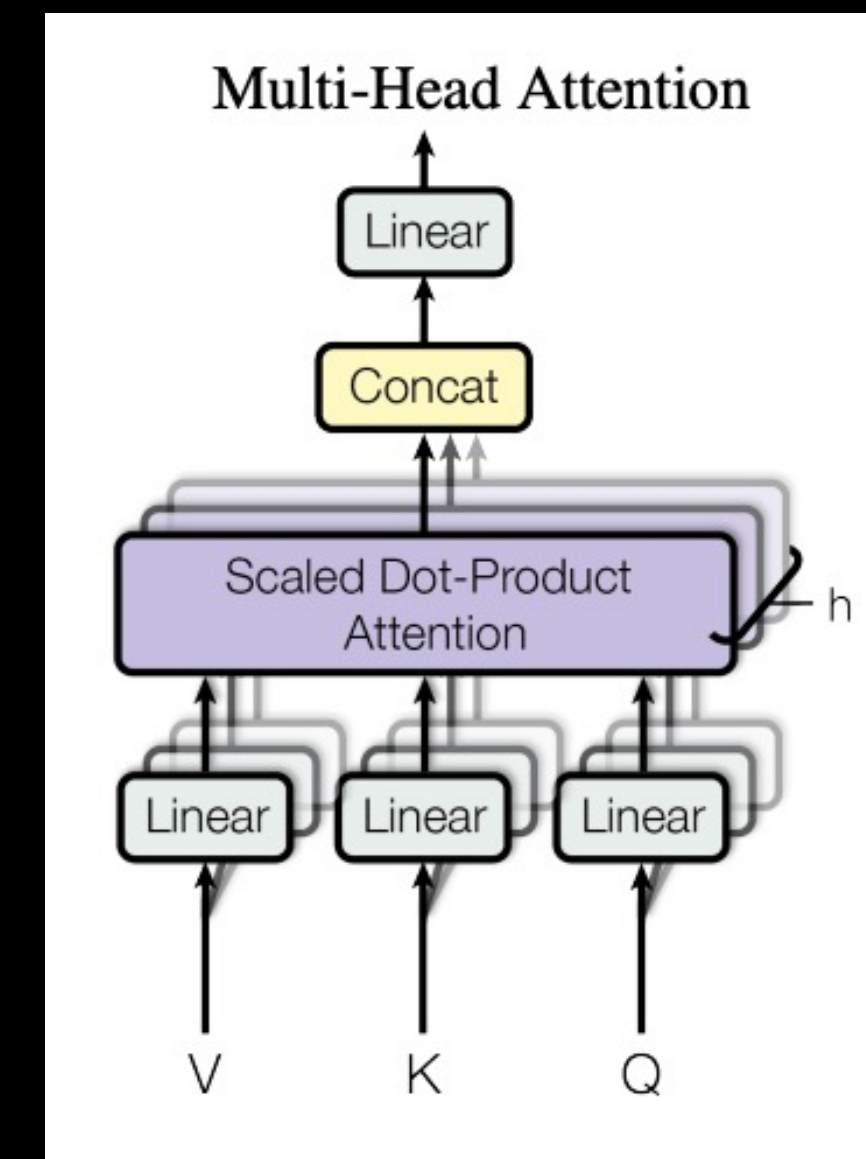
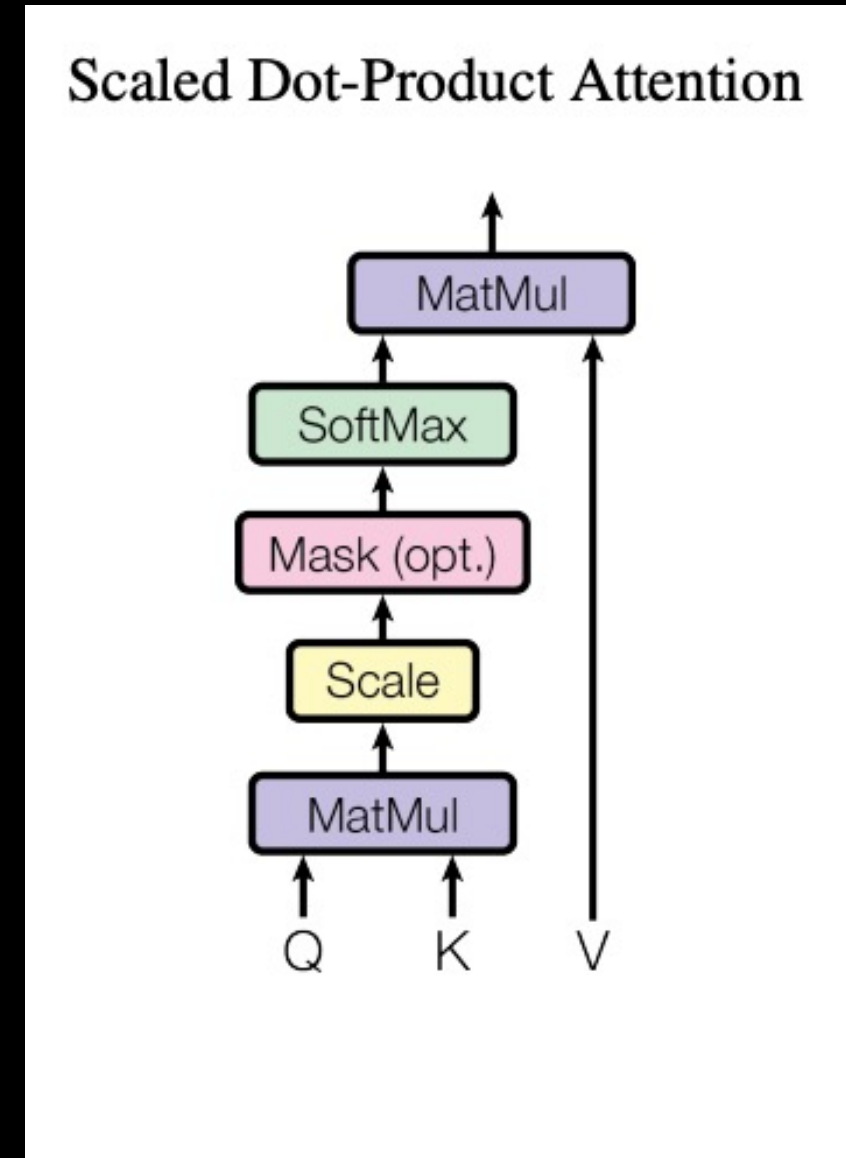


어텐션의 등장

- 원래 기계 번역에서 나온 아이디어
- 문장 전체를 하나의 고정 벡터 입력으로 구성 시 손실이 발생함
- 출력할 때마다 입력 전체를 다시 보는 방식으로 개선

셀프 어텐션 (Q·K·V)

- **셀프 어텐션**
 - 한 시퀀스 안 모든 위치가 **서로를 직접** 바라봄
- **Q·K·V**
 - 질문(Query)·열쇠(Key)·내용(Value)으로 관련도 계산
- 관련도 높은 위치의 내용을 더 많이 **가중합**해서 가져옴
$$\text{softmax}(QK^T / \sqrt{d_k}) V$$



Causal Masking과 인코더

- **Causal Masking** : 다음 정답을 마스킹
- 해당 위치를 음의 무한 적용 -> softmax 후 0이 됨
- 인코더 블록
- 어텐션 + 잔차연결·정규화 + 피드포워드

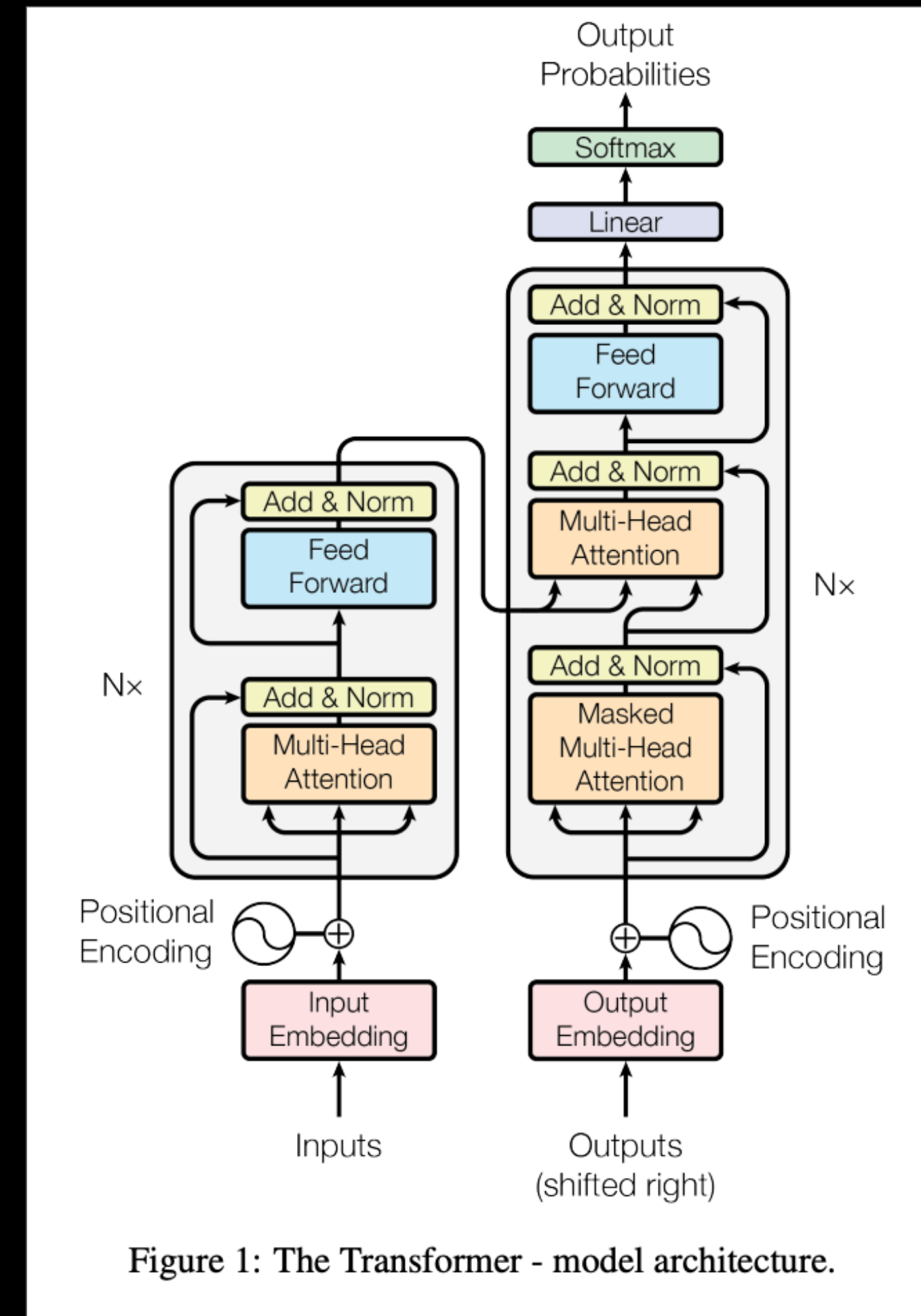
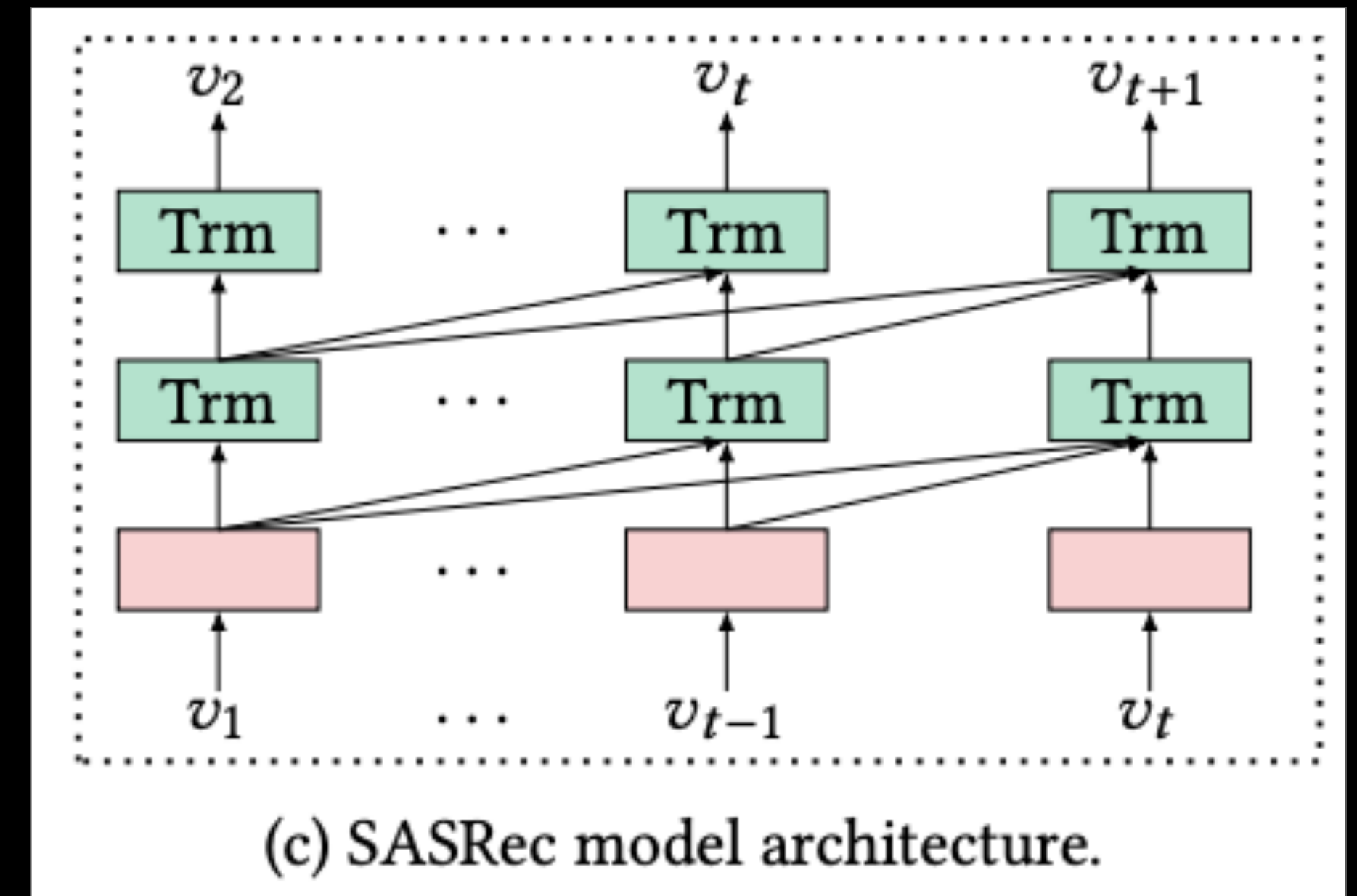


Figure 1: The Transformer - model architecture.

SASRec 구조와 학습

- **아이템 임베딩 + 위치 임베딩(순서 정보) → 셀프 어텐션 → 예측**
- 예측은 **출력 벡터와 아이템 임베딩의 내적**
- 시퀀스를 **한 칸씩 밀어** 다음 아이템을 맞히도록



추천 결과와 어텐션 해석

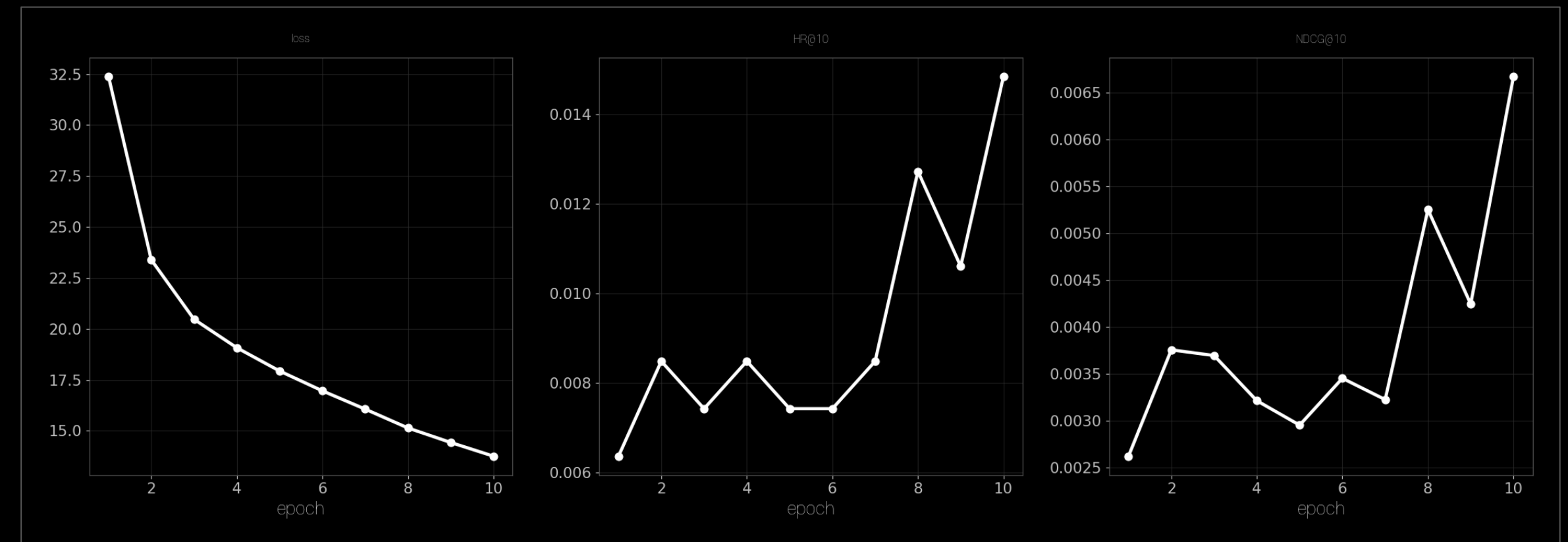
- 마지막 시점 점수에서 다음에 볼 영화를 추천
- CF는 "비슷한 사람/아이템", SASRec은 본 순서의 흐름 → 순서 바뀌면 추천도 달라짐
- 어텐션 히트맵 : 모델이 과거 어느 영화에 집중했는지 눈으로 확인

6주차 Airflow

- 자동화 파이프라인 구축

학습 자동화 및 로깅

- **Airflow** : 학습 과정을 자동 실행 파이프라인(DAG)으로 묶음
- **WandB** : 학습 과정(loss·지표)을 기록·비교 (print와 달리 안 사라짐)
- **leave-one-out** : 마지막 1개를 정답으로 떼어 평가
 - HR@10·NDCG@10로 점수화



3. 관통 개념 정리

- 주차를 가로지르는 핵심

핵심 개념 정리

- 임베딩 : 의미를 벡터로 (4주차 → 5주차 재사용)
- 유사도·내적 : 관계를 숫자로 (4주차 → 5주차 예측)
- 어텐션 : 관련 있는 것에 가중 (5주차 핵심)
- 순차성 : 순서를 반영 (4주차 한계 → 5주차 해결)

개념의 누적

- 각 개념은 한 주차에서 등장해 다음 주차에서 다시 쓰임
- 임베딩 → 유사도 → 어텐션 → 순차성 → 운영, 차곡차곡 쌓임
- 전체가 하나의 추천 시스템을 만들고 굴리는 흐름으로 연결

Q & A

End of Document